



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт кибербезопасности и цифровых технологий
Кафедра КБ-14 «Цифровые технологии обработки данных»

ОТЧЁТ ПО ПРЕДДИПЛОМНОЙ ПРАКТИКЕ

**Тема практики: «Применение обратного обучения с
подкреплением для моделирования поведения агентов в
многоагентной среде»**

приказ Университета о направлении на практику от «21» марта 2026 г. №2617-С

Отчет представлен к
рассмотрению:

«__» _____ 2026 г.

Студент группы: БСМО-20-24

(Подпись) Иванисенко З.И.

Отчет утвержден.
Допущен к защите:

Руководитель практики от кафедры

«__» _____ 2026 г.

(Подпись) Зайцев Е.И.

Москва 2026 г.



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт кибербезопасности и цифровых технологий
Кафедра КБ-14 «Цифровые технологии обработки данных»

ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ НА ПРЕДДИПЛОМНУЮ ПРАКТИКУ

Студенту 2 курса учебной группы БСМО-20-24

Иванисенко Захару Ильичу

Место и время практики: РТУ МИРЭА, кафедра КБ-14 «Цифровые технологии обработки данных», с «23» марта 2026 г. по «16» мая 2026 г.

Должность на практике: студент

1. ЦЕЛЕВАЯ УСТАНОВКА: развитие способностей в области проектной деятельности путем применения современных методов и инструментальных средств прикладной информатики, в том числе путем анализа и моделирования прикладных процессов с учетом выбранной темы исследования.

2. СОДЕРЖАНИЕ ПРАКТИКИ:

2.1. Изучить: исследовать информационные и прикладные процессы.

2.2. Практически выполнить: применить современные инструментальные средства для моделирования информационных и прикладных процессов.

2.3. Ознакомиться: с уровнем развития информационных и прикладных процессов с учетом темы исследования.

3. ДОПОЛНИТЕЛЬНОЕ ЗАДАНИЕ: оформить презентационный материал.

4. ОРГАНИЗАЦИОННО-МЕТОДИЧЕСКИЕ УКАЗАНИЯ: в процессе практики рекомендуется использовать периодические издания и отраслевую литературу годом издания не старше 5 лет.

Заведующий кафедрой:

«__» _____ 2026 г.

(Иванова И.А.)

(подпись)

СОГЛАСОВАНО:

Руководитель практики от кафедры
«23» 03 2026 г.

Зайцев Е.И.

(подпись)

Задание получил
«23» 03 2026 г.

(Иванисенко З.И.)

(подпись)

**Проведенные
инструктажи:**

Охрана труда:

«23» 03 2026 г.

Инструктирующий

Зайцев Е.И., к.т.н., доцент

(подпись)

Инструктируемый

Иванисенко З.И.

(подпись)

Техника безопасности:

«23» 03 2026 г.

Инструктирующий

Зайцев Е.И., к.т.н., доцент

(подпись)

Инструктируемый

Иванисенко З.И.

(подпись)

Пожарная безопасность:

«23» 03 2026 г.

Инструктирующий

Зайцев Е.И., к.т.н., доцент

(подпись)

Инструктируемый

Иванисенко З.И.

(подпись)

С правилами внутреннего распорядка ознакомлен:

«23» 03 2026 г.

Иванисенко З.И.

(подпись)



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

**РАБОЧИЙ ГРАФИК ПРОВЕДЕНИЯ
ПРЕДДИПЛОМНОЙ ПРАКТИКИ**

студента Иванисенко З.И. 2 курса группы БСМО-20-24 очной формы обучения,
обучающегося по направлению подготовки 09.04.02 «Информационные системы и
технологии» программа «Многоагентные интеллектуальные системы»

Неделя	Сроки выполнения	Этап	Отметка о выполнении
1	23.03.2026	Подготовительный этап, включающий в себя организационное собрание (Вводная лекция о порядке организации и прохождения производственной практики, инструктаж по технике безопасности)	
2	26.03.2026	Выполнение задания по практике в соответствии с выданным заданием студента. (Мероприятия по сбору, обработке и структурированию материала, выполнение поставленной задачи)	
3	07.05.2026	Подготовка отчета и презентационного материала по практике (Оформление материалов отчета и презентации в полном соответствии с требованиями на оформление учебных работ студентов)	
4	16.05.2026	Защита отчета по производственной практике у руководителя практики. (Представление отчета по практике к защите)	

Руководитель

практики от кафедры

Обучающийся

_____/Зайцев Е.И./

_____/Иванисенко З.И./

Согласовано:

Заведующий кафедрой

_____/Иванова И.А., к.т.н., доцент/

ОТЧЁТ

по преддипломной практике

студента 2 курса учебной группы БСМО-20-24 Института КБ

Иванисенко Захара Ильича

1. Практику проходил с 23.03.2026 г. по 16.05.2026 г. в ФГБОУ ВО «МИРЭА – Российский технологический университет», на кафедре КБ-14 «Цифровые технологии обработки данных», студент

(место прохождения практики и должность)

2. Задание на практику выполнил
в полном объеме/частично

(нужное подчеркнуть)

Не выполнены следующие задания:

(указать также причины невыполнения)

Подробное содержание выполненной на практике работы и достигнутые результаты:
проведено исследование прикладной области в части изучения

Предложения по совершенствованию организации и прохождения практики:
предложений нет

Студент _____ (Иванисенко З.И.)

(подпись)

«__» _____ 2026 г

Заключение руководителя практики

Приобрел следующие профессиональные навыки: студент продемонстрировал профессиональные умения и навыки, знание и понимание прикладной области, задач, требующих решения в прикладной области, современные подходы и средства решения прикладных задач разных классов, умение находить и работать с различными источниками информации по профессиональной деятельности, структурировать отчет с учетом тематики исследования

Проявил себя как: студент соблюдал/не соблюдал сроки календарного графика практики.

(нужное подчеркнуть)

«__» _____ 2026 г.

Отчет проверил:

Руководитель практики от Университета

_____ (Зайцев Е.И.)

(подпись)

Содержание

Введение	3
2. Структурная и функциональная организация системы моделирования	5
2.1 Архитектура системы	5
2.1.1 Описание исходной одноагентной симуляции дорожного движения	5
2.1.2 Переход к многоагентной архитектуре и принцип локальной наблюдаемости	7
2.2 Алгоритмическое и программное обеспечение	9
2.2.1 Реализация структуры среды	9
2.2.2 Реализация временных характеристик среды	11
2.2.3 Реализация агентной маршрутизации	13
2.3 Интерфейс пользователя	16
Таблица 2.1 Ползунки параметров среды	20
3. Реализация системы и оценка результатов	22
3.1 Технологии разработки	22
3.1.1 Симуляционный цикл, визуализация и случайное изменение среды	22
3.1.2 Механизм логирования и формат фиксируемых данных	26
3.2 Обучение и внедрение модели	29
3.2.1 Исходные данные и дискретизация пространства состояний	29
3.2.2 Построение модели переходов и вычисление меры посещаемости	32
3.2.3 Обучение функции вознаграждения и извлечение политики	34
3.2.4 Сохранение и применение модели	37
3.2.5 Интеграция агента в систему и замена ручного управления	38
3.2.6 Загрузка модели и инференс в симуляционном цикле	40
3.2.7 Аппроксимация будущего состояния и отображение решения в интерфейсе	42
3.3 Оценка результатов экспериментов	44
Заключение	47
Список литературы	49

ВВЕДЕНИЕ

Современные многоагентные системы находят применение в широком спектре задач — от управления транспортными потоками до распределения вычислительных ресурсов. Общей чертой таких систем является высокая динамичность среды и необходимость координации между автономными агентами, каждый из которых обладает лишь частичной информацией о глобальном состоянии. Статичные, заранее запрограммированные стратегии управления в подобных условиях оказываются неэффективными, что порождает потребность в адаптивных интеллектуальных методах.

Обучение с подкреплением (Reinforcement Learning) является одним из перспективных подходов к решению данной задачи. Однако его практическое применение сдерживается необходимостью явного задания функции вознаграждения, которая в сложных многоагентных средах зачастую неизвестна или трудноформализуема. Обратное обучение с подкреплением (Inverse Reinforcement Learning, IRL) позволяет восстановить функцию вознаграждения по наблюдаемому поведению эксперта, что делает данный метод применимым в ситуациях, когда корректные стратегии управления могут быть продемонстрированы, но не формализованы аналитически.

Целью настоящей работы является разработка системы моделирования поведения агентов в многоагентной среде методом обратного обучения с подкреплением. Для достижения поставленной цели решаются следующие задачи: исследование предметной области и обоснование выбора метода IRL; проектирование архитектуры многоагентной системы с учётом частичной наблюдаемости и взаимозависимости агентов; программная реализация системы и экспериментальная оценка полученных результатов.

В качестве демонстрационной среды используется симуляция упрощённой транспортной сети, в которой агенты управляют распределением автомобильного потока между полосами движения, принимая решения на

основе локальной информации и обмениваясь сигналами с соседними агентами.

В данной практической работе предоставлены разделы 2 и 3 будущей диссертации.

2. СТРУКТУРНАЯ И ФУНКЦИОНАЛЬНАЯ ОРГАНИЗАЦИЯ СИСТЕМЫ МОДЕЛИРОВАНИЯ

2.1 Архитектура системы

2.1.1 Описание исходной одноагентной симуляции дорожного движения

В качестве демонстрационного стенда рассмотрена упрощённая, но концептуально репрезентативная система из нескольких взаимосвязанных транспортных полос, объединённых интеллектуальным балансировщиком трафика.

Для исследования методов обратного обучения с подкреплением была разработана веб-симуляция на Python, реализующая упрощённую модель управления транспортными потоками на участке дорожной сети. Симуляция служит контролируемой средой, позволяющей изолировать ключевые свойства многоагентного взаимодействия без излишней сложности полноценных транспортных систем. Упрощённые среды обеспечивают воспроизводимость экспериментов, управляемость параметров и возможность строгой верификации алгоритмов.

Топология среды устроена следующим образом. Входящий поток движется по главной полосе (`main_line`), которая разветвляется на две дополнительные полосы (`highway1` и `highway2`). Автомобили организованы в три независимые FIFO-очереди на основе структуры `deque`. Ёмкость главной полосы составляет 12 транспортных единиц, каждой дополнительной — 10 единиц. Разница в ёмкости отражает типичную ситуацию, когда магистраль обладает большей пропускной способностью по сравнению с ответвлениями.

Динамика системы определяется четырьмя временными параметрами (в секундах): `main_delay` — задержка подачи новых автомобилей в главную полосу, регулирующая интенсивность входящего потока; `transfer_delay` — время перевода автомобиля из главной полосы в дополнительную,

характеризующее скорость маршрутизации; `drain_delay_h1` и `drain_delay_h2` — пропускные способности на выходе из первой и второй дополнительных полос соответственно.

Все перечисленные параметры волатильны: их значения могут изменяться динамически в процессе работы системы. Агент не может опираться на статичную модель мира и вынужден адаптироваться к текущим условиям. Изменение задержек влияет на соотношение скоростей накопления и освобождения очередей, создавая нетривиальную задачу управления: в зависимости от актуальных значений `drain_delay_h1` и `drain_delay_h2` оптимальное направление маршрутизации может кардинально меняться.

Пространство состояний среды включает: текущую заполненность каждой из трёх очередей, актуальные значения четырёх временных параметров, а также сведения о достижении какой-либо очередью максимальной ёмкости. Эти характеристики формируют наблюдаемое состояние, на основе которого агент принимает решение.

Управляющий параметр в одноагентной постановке — целочисленный `line_choose`, принимающий значение 1 или 2 в зависимости от выбранной дополнительной полосы. Пространство действий бинарно и глобально: агент единолично решает о маршрутизации для всей системы, располагая полным наблюдением за состоянием всех очередей и параметрами среды.

Это обстоятельство обнажает принципиальное ограничение одноагентной архитектуры. Управление транспортными потоками в реальных системах предполагает децентрализованное принятие решений: каждый узел обрабатывает локальную информацию и взаимодействует с соседними через ограниченные каналы коммуникации. Одиночный агент с глобальным наблюдением не позволяет воспроизвести ни частичную наблюдаемость, ни конкурирующие интересы, ни эмерджентное коллективное поведение. Кроме того, алгоритмы обратного обучения с подкреплением направлены на восстановление функций вознаграждения отдельных агентов по их поведению, что лишено смысла при единственном централизованном

управляющем. Следовательно, одноагентная симуляция представляет лишь отправную точку, требующую архитектурной трансформации для многоагентного исследования.

2.1.2 Переход к многоагентной архитектуре и принцип локальной наблюдаемости

Для приведения симуляции в соответствие с задачами многоагентного обучения с подкреплением была проведена реструктуризация: вместо единого агента введены два независимых агента, каждый закреплён за одной из дополнительных полос — highway1 и highway2. Это перевело систему из класса MDP (Markov Decision Process) в класс POSG (Partially Observable Stochastic Game), что является теоретической основой для применения многоагентных методов.

Ключевой принцип новой архитектуры — информационная изоляция агентов. Каждый агент имеет доступ исключительно к локальному состоянию своей полосы: текущей заполненности очереди, параметру drain_delay и собственному сигналу готовности. Состояние главной полосы и параметры противоположной полосы остаются вне наблюдаемого пространства. Формально, если полное состояние системы $s \in S$, то агент i наблюдает лишь проекцию $o_i = \Omega_i(s)$, где Ω_i — функция частичного наблюдения, ограниченная параметрами i -й полосы.

Пространство действий каждого агента также изменилось. Если в исходной постановке line_choose описывал глобальный выбор направления, то в многоагентной архитектуре каждый агент оперирует локальным бинарным решением: can_take = 1 означает готовность принять автомобиль, 0 — отказ. Пространство действий каждого агента $A_i = \{0, 1\}$, а совместное пространство $A = A_1 \times A_2$ описывает четыре комбинации сигналов.

Логика маршрутизации определяется комбинацией сигналов обоих агентов: при can_take1 = 0 и can_take2 = 0 автомобиль остаётся в главной полосе (режим ожидания); при обоюдной готовности выбор осуществляется

случайно с вероятностью 0.5; при готовности только одного агента автомобиль направляется на его полосу. Схема реализует неявную координацию через совместный исход независимых решений.

Коммуникация между агентами намеренно ограничена бинарными сигналами готовности `can_take`. Коммуникативный канал $C = \{0, 1\}$ несёт один бит на агента за шаг взаимодействия, что соответствует модели коммуникации с ограниченной пропускной способностью, характерной для децентрализованных систем реального времени.

Важная особенность архитектуры — асимметрия агентов. Агент `highway1` в прототипе реализован с ручным управлением: `can_take1` задаётся экспериментатором или обучаемым алгоритмом. Агент `highway2` реализован с пороговой эвристикой: `can_take2 = 1`, если заполненность `highway2` не превышает порога, иначе `can_take2 = 0`. Поведение автоматического агента формирует траектории-демонстрации для алгоритмов обратного обучения с подкреплением, восстанавливающих неявную функцию вознаграждения.

Пороговая стратегия второго агента порождает нетривиальную динамику взаимодействия. При высоком пороге второй агент почти всегда готов к приёму, и первый агент конкурирует с ним за транспортные единицы. При низком пороге второй агент часто отказывает, и первый становится единственным доступным направлением. Варьирование порога создаёт спектр режимов — от конкуренции до монопольного обслуживания, — обеспечивая разнообразие контекстов в экспертных демонстрациях и повышая обобщающую способность восстановленной функции вознаграждения.

Частичная наблюдаемость, бинарное пространство действий, ограниченная коммуникация и асимметрия агентов формируют среду с ключевыми свойствами децентрализованного POSG, пригодную для исследования алгоритмов обратного обучения с подкреплением в многоагентном контексте. Переход от одноагентной к многоагентной постановке ставит фундаментальный вопрос о возможности восстановления

индивидуальных функций вознаграждения по наблюдаемым траекториям в условиях взаимных зависимостей и неполноты информации.

2.2 Алгоритмическое и программное обеспечение

2.2.1 Реализация структуры среды

Симуляционная среда построена на основе формальной модели дорожной топологии с разветвлением потока. Топология представляет участок дороги, где единственная главная полоса разделяется на две независимые дополнительные — типичный сценарий для анализа распределения потока и поведения агентов при маршрутных решениях.

Дорожная сеть формализована в виде трёх очередей с ограниченной ёмкостью: первая соответствует главной полосе, вторая и третья — альтернативным направлениям после точки разветвления. Главная полоса обладает большей вместимостью, что соответствует реальному соотношению пропускных способностей: магистральный въезд рассчитан на суммарный поток, каждое разветвление принимает лишь его часть.

Каждый автомобиль в очереди кодируется как атомарный элемент без дополнительных атрибутов. Такое упрощение обоснованно: индивидуальные характеристики транспортного средства не существенны для моделирования агрегированного поведения потока на уровне маршрутных решений. Среда оперирует характеристиками заполненности участков — плотностью потока.

Формально среда представляет собой MDP, определяемый кортежем $\langle S, A, T, R \rangle$. Пространство состояний S образуется тройкой длин очередей $(|main|, |h1|, |h2|)$ в совокупности с параметрами задержек. Ключевое свойство — выполнение марковского условия: зная текущие длины очередей и значения задержек, будущая эволюция системы полностью определяется настоящим состоянием и не зависит от предшествующей истории. Это оправдывает применение аппарата MDP и обеспечивает теоретическую корректность обучения методами обратного обучения с подкреплением.

Очереди используют дисциплину обслуживания FIFO: автомобиль, въехавший на участок ранее, покидает его раньше, что точно отражает физику дорожного движения в нормальных условиях. Модель FIFO обеспечивает корректное моделирование эффекта насыщения: при заполнении очереди до предельной ёмкости новые элементы не добавляются, воспроизводя образование пробки. Этот механизм ключевой для изучения поведения агентов в условиях перегрузки: именно тогда ценность маршрутного выбора наиболее значима.

Ограничения вместимости влияют на информативность сигнала вознаграждения, восстанавливаемого IRL-алгоритмом. По мере приближения очереди к предельной заполненности цена ошибочного решения резко возрастает: направление автомобиля на переполненное шоссе блокирует поток, тогда как выбор менее загруженного маршрута сохраняет пропускную способность. Поведение эксперта вблизи предела заполнения наиболее полно раскрывает структуру его неявной функции вознаграждения, поскольку различные стратегии дают существенно расходящиеся долгосрочные исходы. Механизм насыщения выполняет двойную роль: моделирует реалистичные условия транспортного коллапса и обеспечивает экспертным траекториям высокую различающую способность для точного восстановления функции вознаграждения.

Транспортный поток организован как последовательное прохождение через три функциональных зоны: внешний источник, генерирующий транспортные единицы; главная полоса, выполняющая функцию буфера перед точкой маршрутного выбора; две дополнительные полосы, обеспечивающие выход из системы.

Три структуры данных изолированы друг от друга: информация о заполненности одной полосы не передаётся в другую. Взаимодействие между участками осуществляется исключительно через логику маршрутизации агентов. Тем самым достигается разделение между пассивной структурой

среды и активной логикой принятия решений, что соответствует принципу разграничения среды и агента в формализме MDP.

Соответствие модели реальным транспортным процессам обеспечивается на нескольких уровнях: топология воспроизводит типичный сценарий разветвления потока; механизм FIFO отражает последовательное продвижение автомобилей; конечная ёмкость моделирует физическое пространство дороги и формирует явление перегрузки.

Ограниченная ёмкость выполняет двойную функцию в контексте обучения с подкреплением: формирует ограничения среды для принятия решений и составляет наблюдаемое состояние, на основании которого агенты формируют политику поведения. Структура данных среды одновременно задаёт динамику системы и информационную основу для агентного обучения.

Предложенная архитектура обеспечивает достаточную степень абстракции реального транспортного процесса, сохраняя ключевые свойства — конечность ресурсов, последовательность обслуживания и топологическую взаимосвязь участков, — что делает её пригодной для применения методов обратного обучения с подкреплением.

2.2.2 Реализация временных характеристик среды

Воспроизведение динамики транспортного потока требует явного задания временных характеристик — параметров интенсивности и пропускной способности каждого элемента дорожной сети. В среде эти характеристики реализованы как конфигурируемые задержки событий.

В основе временной архитектуры лежит принцип дискретно-событийного моделирования: симуляционный цикл выполняется с фиксированным шагом порядка нескольких десятков миллисекунд, однако транспортные события — появление автомобиля, перемещение между полосами, выезд из системы — происходят лишь при выполнении временного условия. На каждой итерации для каждого класса событий проверяется, истёк

ли установленный интервал задержки; при выполнении условия событие генерируется и счётчик сбрасывается.

Такой подход совмещает непрерывность модельного времени с дискретностью транспортных событий, что соответствует классической схеме систем массового обслуживания с интенсивностью входящего потока λ и скоростями обслуживания μ .

Конфигурация включает четыре временных параметра. Задержка подачи в основную полосу определяет интервал поступления новых транспортных средств — аналог интенсивности входящего потока в теории очередей. Диапазон значений перекрывает режимы от интенсивного городского трафика до редкого загородного потока.

Задержка перевода задаёт темп перемещения автомобилей из основной полосы в дополнительную, моделируя пропускную способность точки принятия решения агентом. При малых значениях агент оперативно перераспределяет поток; при высоких — управление становится инерционным.

Задержки выезда из первой и второй полос описывают пропускные способности выходов сети. Параметры независимы, что позволяет моделировать асимметричные конфигурации — например, когда одна полоса ведёт к скоростному шоссе, а другая — к местной дороге с ограниченным выездом.

Все четыре параметра волатильны: они могут изменяться в реальном времени через пользовательский интерфейс. Технически это реализовано через разделяемый конфигурационный объект с синхронизацией посредством механизма взаимного исключения, исключающего состояние гонки между симуляционным циклом и потоком интерфейса.

Комбинация параметров порождает богатое пространство конфигураций. При малом `main_delay` и больших `drain_delay` автомобили поступают быстрее, чем покидают шоссе, создавая давление дефицита ресурса. При значительной разнице `drain_delay_h1` и `drain_delay_h2`

оптимальная стратегия явно предпочитает одно из шоссе. Комбинаторное разнообразие конфигураций обеспечивает широкий спектр контекстов принятия решений — от тривиальных до критических. Это является необходимым условием для восстановления обобщающей функции вознаграждения: чем репрезентативнее демонстрации эксперта, тем точнее аппроксимируется его целевая функция.

Такая архитектура позволяет записывать экспертные траектории при одном наборе параметров, а обучение и тестирование проводить при другом, что даёт возможность исследовать обобщающую способность восстановленной функции вознаграждения.

Наряду с временными параметрами среда задаёт фиксированные структурные ограничения — максимальные ёмкости очередей. Основная полоса допускает больше автомобилей, чем каждая дополнительная, что отражает типичную архитектуру транспортных узлов. При достижении максимальной ёмкости поступление новых автомобилей блокируется, моделируя физические ограничения дорожного пространства.

Совокупность временных параметров и структурных ограничений полностью определяет динамику среды. Варьируя параметры, исследователь формирует различные режимы нагрузки для сбора экспертных демонстраций и оценки качества восстановленной политики.

2.2.3 Реализация агентной маршрутизации

Подсистема маршрутизации представляет собой децентрализованный механизм принятия решений, в котором каждый агент функционирует в условиях частичной наблюдаемости и взаимодействует с остальными участниками посредством локальных управляющих сигналов. Ключевой принцип — разделение информационного пространства: ни один агент не располагает полным знанием о состоянии сети и принимает решения на основе наблюдений, относящихся к зоне его ответственности.

Каждый агент обслуживает отдельную въездную полосу и опирается только на локальную информацию — параметры состояния собственной очереди и пропускную способность участка. Глобальное состояние потока агенту недоступно. Такая архитектура соответствует постановке POSG, в которой агент располагает лишь локальным наблюдением. Ограничение наблюдаемости является намеренным проектным решением, отражающим реальные условия распределённых транспортных систем.

Два агента обладают принципиально различными механизмами формирования решений — это осознанный методологический выбор. Первый агент функционирует в режиме ручного управления: оператор задаёт его решение о готовности принять транспортное средство. Действия этого агента фиксируются и формируют демонстрационные траектории для методов обратного обучения с подкреплением, позволяя накапливать корпус экспертного поведения без формальной спецификации функции вознаграждения.

Второй агент реализует детерминированную пороговую стратегию: готовность определяется соотношением текущей длины очереди и порогового значения. Порог является настраиваемым параметром, позволяющим варьировать нагрузку на вторую полосу и обеспечивать разнообразие сценариев.

Такая асимметрия подготавливает систему к ключевому переходу: на завершающем этапе ручное управление первым агентом заменяется моделью, обученной методами обратного обучения с подкреплением на накопленных демонстрациях. Это позволяет количественно верифицировать качество воспроизведения экспертного поведения.

Координация агентов осуществляется через коммуникационный канал: каждый агент транслирует бинарный сигнал готовности — один бит на агента за временной шаг. Этот подход соответствует парадигме децентрализованного исполнения при централизованном агрегировании сигналов: агенты действуют автономно, а итоговое решение о маршрутизации формируется

координирующим модулем на основе совокупности сигналов. Агенты не имеют доступа ни к наблюдениям, ни к стратегиям друг друга.

Координирующий модуль реализует детерминированную таблицу маршрутизации для всех четырёх комбинаций входных сигналов.

При взаимной неготовности обоих агентов система переходит в режим ожидания — ни одно транспортное средство не направляется ни на одну полосу, что предотвращает накопление заторов.

При взаимной готовности целевая полоса выбирается равновероятно ($p = 0.5$), обеспечивая статистически равномерное распределение нагрузки и исключая систематический дисбаланс очередей.

При исключительной готовности одного агента транспортное средство однозначно направляется на его полосу, исключая неопределённость при недоступности альтернативы.

Описанная схема обладает существенными свойствами: полнота покрытия — таблица определена для всех комбинаций входных сигналов; минимальность коммуникации — для координации достаточно одного бита от каждого агента; масштабируемость — механизм может быть расширен при увеличении числа агентов, хотя пространство совместных состояний растёт экспоненциально.

Данная реализация создаёт воспроизводимую экспериментальную среду для накопления демонстрационных данных с контролируемыми параметрами нагрузки — необходимое условие применения алгоритмов обратного обучения с подкреплением. Вариативность порогового параметра второго агента позволяет систематически изменять условия наблюдения за первым агентом, обеспечивая разнообразие и репрезентативность обучающей выборки.

2.3 Интерфейс пользователя

Диаграмма прецедентов, представленная на рисунке 1, описывает взаимодействие оператора с графическим интерфейсом симуляционной системы многоагентного взаимодействия.



Рисунок 2. 1 – Диаграмма прецедентов

Главным действующим лицом является **оператор** — пользователь, взаимодействующий с системой через веб-браузер. Выполняет роль эксперта: управляет симуляцией, задаёт параметры среды, принимает решения о маршрутизации транспортных единиц и фиксирует свои действия для последующего обучения модели.

Прецеденты описанные на диаграмме:

1. **Управлять симуляцией** — оператор запускает, приостанавливает или сбрасывает симуляционный цикл при помощи кнопок «Старт», «Стоп» и

«Сброс». Сброс очищает все транспортные очереди и возвращает систему в исходное состояние.

2. **Настроить параметры транспортной среды** — оператор регулирует динамические характеристики среды через ползунки: интенсивность входящего потока, скорость перевода между полосами, пропускную способность каждого из двух альтернативных маршрутов, а также пороговое значение заполненности для автоматического агента. Изменения применяются мгновенно в реальном времени.

3. **Наблюдать состояние транспортных потоков** — центральная панель отображает три полосы движения в виде цветowych ячеек с числовыми счётчиками заполненности. Визуализация обновляется в реальном времени после каждого шага симуляции. Также отображаются статусные бейджи работы симуляции и записи данных.

4. **Принять решение о маршрутизации** — оператор переключает готовность первой полосы к приёму автомобиля (кнопка-переключатель). Это ключевое экспертное действие, формирующее обучающую выборку. Готовность второй полосы вычисляется автоматически по пороговому правилу и отображается в виде пассивного индикатора. В финальной версии системы данное решение принимается обученной моделью автоматически, а интерфейс отображает результат в режиме «только чтение» с указанием вероятности принятого решения.

5. **Записать экспертные демонстрации** — оператор активирует и завершает запись состояния среды. Система фиксирует наблюдаемые параметры и решения оператора с интервалом в одну секунду и сохраняет результат в файл.

6. **Рандомизировать параметры среды** — оператор нажимает кнопку, генерирующую случайные значения для всех параметров среды в допустимых диапазонах. Прецедент расширяет настройку параметров, обеспечивая разнообразие экспериментальных сценариев и покрытие пространства состояний в демонстрациях.

Связи между прецедентами:

- «Рандомизировать параметры среды» расширяет (<<extend>>) «Настроить параметры транспортной среды»: рандомизация является альтернативным способом задать конфигурацию вместо ручной настройки каждого ползунка.
- «Принять решение о маршрутизации» включает (<<include>>) «Наблюдать состояние транспортных потоков»: решение о готовности полосы принимается на основании визуальной информации о заполненности очередей.
- «Записать экспертные демонстрации» включает (<<include>>) «Наблюдать состояние транспортных потоков»: в процессе записи оператор непрерывно отслеживает состояние системы для принятия обоснованных решений.

Сам по себе интерфейс представляет собой одностраничное веб-приложение с тёмной цветовой темой, представленное на рисунке 2.2.



Рисунок 2. 2 – Симуляция

Компоновка вертикальная, элементы центрированы по горизонтали. Интерфейс разделён на три визуальных уровня сверху вниз:

- (А) Сцена дорожной сети — визуализация очередей
- (В) Строка счётчиков и статусных бейджей
- (С) Панель управления — кнопки, ползунки, индикаторы

Цветовое кодирование единообразно во всех элементах представлено в таблице 2.1

Таблица 2.1 – Цвета на визуализации

Цвет	Назначение
Бирюзовый	Главная полоса
Пурпурный	Первая дополнительная полоса
Зелёный	Вторая дополнительная полоса
Оранжевый	Кнопка рандомизации
Фиолетовый	Индикатор модели и логирования

Горизонтальный контейнер, разделённый на три зоны:

- **Левая зона — главная полоса.** Горизонтальная полоса из 12 ячеек. Занятые ячейки окрашиваются в бирюзовый с эффектом свечения, пустые остаются тёмно-серыми.

- **Центральная зона — развилка.** Вертикальный разделитель со стрелочными индикаторами, обозначающий точку разветвления потока.

- **Правая зона — два маршрута.** Два горизонтальных блока друг над другом: первая полоса (10 пурпурных ячеек) и вторая полоса (10 зелёных ячеек). При отказе второй полосы от приёма её блок подсвечивается красным ореолом.

Визуализация обновляется в реальном времени: сервер транслирует состояние всех очередей после каждой итерации, клиент перерисовывает ячейки с плавной анимацией.

Строка счётчиков и статусных бейджей (средняя часть) представляет собой горизонтальную строку, содержащую:

- Три числовых счётчика заполненности очередей в формате «N / max» для каждой из полос.
- Бейдж состояния симуляции: «РАБОТАЕТ» (бирюзовый) или «СТОП» (пурпурный).
- Бейдж состояния логирования (в прототипе): «ЗАПИСЬ» (фиолетовый) или «ВЫКЛ» (серый).

Панель управления (нижняя часть) представлена темной панелью с элементами управления, сгруппированными в блоки.

Управление симуляцией происходит посредством четырех кнопок в горизонтальном ряду:

- **«Старт»** (бирюзовая) — запуск симуляционного цикла;
- **«Стоп»** (пурпурная) — остановка цикла;
- **«Сброс»** (серая) — очистка очередей и остановка;
- **«Random»** (оранжевая) — случайная генерация значений всех

параметров среды в допустимых диапазонах.

Логирование данных (только прототип) происходит через 2 кнопки:

- **«Начать лог»** (фиолетовая) — запуск записи состояния среды;
- **«Сохранить лог»** — завершение записи и сохранение файла.

При сохранении в углу экрана появляется уведомление с именем файла.

Индикаторы и решения маршрутизации:

В прототипе: кнопка-переключатель готовности первой полосы (зелёная при разрешении, серая при запрете). Оператор нажимает для изменения решения.

В финальной версии: пассивный индикатор, отображающий бинарное решение обученной модели (0 или 1), метку «MCE IRL» и вероятность решения в процентах. Элемент доступен только для чтения.

Индикатор готовности второй полосы: пассивная цветовая точка (зелёная — готова, пурпурная — не готова), вычисляемая автоматически по пороговому правилу.

Ползунок порога второй полосы (1–10): определяет уровень заполненности, при превышении которого полоса отказывает в приёме.

Четыре ползунка, каждый с подписью и отображением текущего значения представлены в таблице 2.1

Таблица 2.1 Ползунки параметров среды

Параметр

Диапазон Цвет акцента

Интенсивность входящего потока	0.2–4.0 с	Бирюзовый
Скорость маршрутизации	0.1–4.0 с	Бирюзовый
Пропускная способность первой полосы	0.1–8.0 с	Пурпурный
Пропускная способность второй полосы	0.1–8.0 с	Зелёный

Изменение любого ползунка мгновенно применяется к работающей симуляции. Значения синхронизируются между сервером и всеми подключёнными клиентами в реальном времени.

3. РЕАЛИЗАЦИЯ СИСТЕМЫ И ОЦЕНКА РЕЗУЛЬТАТОВ

3.1 Технологии разработки

В рамках данной работы разработана программная среда для симуляции многоагентного взаимодействия в условиях управляемых транспортных потоков. Реализация системы опирается на совокупность взаимосвязанных программных механизмов, каждый из которых отвечает за отдельный аспект функционирования симуляционной платформы: организацию и исполнение симуляционного цикла, визуализацию текущего состояния среды в реальном времени, генерацию случайных изменений параметров окружения, а также сбор и сохранение данных о поведении экспертного агента. Совокупность указанных механизмов образует единую технологическую основу, обеспечивающую воспроизводимость экспериментов, качество обучающих данных и интерпретируемость результатов обратного обучения с подкреплением.

3.1.1 Симуляционный цикл, визуализация и случайное изменение среды

Симуляционный цикл представляет собой центральный управляющий компонент программной среды, определяющий динамику изменения состояния транспортной сети во времени. В основу архитектуры положена клиент-серверная модель взаимодействия, реализованная посредством веб-фреймворка Flask в сочетании с библиотекой Flask-SocketIO, обеспечивающей двунаправленную передачу данных между сервером и браузерным клиентом по протоколу WebSocket. Режим работы серверного цикла событий установлен как потоковый, что позволяет обрабатывать несколько одновременных соединений без применения асинхронных сопрограмм, упрощая управление разделяемым состоянием симуляции.

Внутренняя топология транспортной сети моделируется тремя очередями типа FIFO с фиксированной вместимостью. Основная очередь, представляющая собой входной буфер транспортных единиц, допускает одновременное нахождение до двенадцати объектов. Две дочерние очереди — первая и вторая полосы движения — имеют ёмкость по десять позиций каждая. Данная топология воспроизводит типичную схему автомобильного перекрёстка или развилки, на которой диспетчер должен принимать решение о направлении транспортного потока в зависимости от загруженности каждого из возможных маршрутов.

Итерационный цикл симуляции выполняется в отдельном фоновом потоке исполнения, не блокирующем основной серверный процесс. На каждой итерации последовательно проверяются четыре временных условия, каждое из которых управляется собственным конфигурируемым параметром задержки. Первое условие регулирует интенсивность поступления транспортных единиц во входную очередь: при истечении заданного интервала к основной очереди добавляется новый объект при наличии свободного места. Второе условие управляет переносом транспортной единицы из входной очереди в одну из двух полос движения согласно логике маршрутизации, определяемой бинарными флагами доступности каждого направления. Третье и четвёртое условия контролируют скорость выбытия транспортных единиц из первой и второй полос движения соответственно, имитируя разные пропускные способности альтернативных маршрутов.

Разделение управляющих задержек по каждому участку сети позволяет моделировать широкий спектр режимов нагрузки: от равномерного распределения потоков до критических ситуаций с образованием заторов. Временные параметры допускают независимую регулировку, что позволяет создавать разнообразные сценарии, охватывающие как типовые, так и экстремальные условия эксплуатации транспортной инфраструктуры.

После завершения каждой итерации серверный компонент транслирует текущее состояние симуляции всем подключённым клиентам посредством

широковещательной рассылки. Передаваемое сообщение содержит сериализованное представление состояния всех трёх очередей в виде массивов фиксированной длины с дополнением нулевыми значениями, текущие счётчики объектов в каждом узле, значения конфигурационных параметров и состояние флагов маршрутизации. Использование массивов фиксированной длины обеспечивает унифицированный формат данных вне зависимости от фактической заполненности очередей.

Клиентская часть системы реализована в виде интерактивной веб-страницы, получающей обновления состояния через механизм подписки на серверные события. Полученные данные отражаются в визуальном интерфейсе в виде цветовых индикаторов, соответствующих заполненным и пустым позициям каждой очереди. Операторы системы могут наблюдать динамику изменения транспортных потоков в реальном времени, отслеживая образование и рассасывание очередей на всех участках моделируемой сети. Для предотвращения избыточной загрузки процессора между итерациями цикла вводится короткая пауза, обеспечивающая сбалансированное использование вычислительных ресурсов без ущерба для воспринимаемой плавности визуализации.

Принципиально важным архитектурным решением является полное разделение симуляционного ядра и слоя визуализации. Вся логика симуляции — операции над очередями, решения о маршрутизации, проверки условий задержки — инкапсулирована в серверном потоке и функционирует независимо от наличия подключённых клиентов. Визуализация выступает пассивным наблюдателем: получает снимки состояния через широковещательную рассылку и не оказывает обратного воздействия на ход симуляции. Такое разделение обеспечивает детерминированность и воспроизводимость результатов вне зависимости от активности пользовательского интерфейса. Данное свойство имеет критическое значение для научной воспроизводимости обучающих данных IRL: траектории, собранные в сеансах без визуализации, идентичны траекториям, полученным

в полноэкранном режиме, что исключает влияние побочных эффектов рендеринга на обучающую выборку.

Получение качественных обучающих данных для алгоритмов обратного обучения с подкреплением требует разнообразия наблюдаемых траекторий экспертного поведения. Если обучающий агент неизменно действует в одинаковых условиях, извлечённая функция вознаграждения будет специфична для узкого набора сценариев и не обладать достаточной обобщающей способностью. Для преодоления данного ограничения в состав среды включён механизм случайного изменения параметров окружения, реализованный в виде интерактивного элемента управления.

При активации механизма клиентская часть производит независимую равномерную выборку значений для каждого из пяти конфигурационных параметров в пределах установленных допустимых диапазонов. Параметр интенсивности входящего потока принимает значения из непрерывного диапазона с заданным шагом дискретизации, позволяя воспроизводить как разреженные, так и насыщенные режимы поступления транспортных единиц. Параметр задержки передачи, управляющий скоростью перераспределения объектов между входной очередью и полосами движения, охватывает диапазон, допускающий как практически мгновенное, так и значительно замедленное перенаправление потока. Параметры скорости разгрузки для каждой из двух полос движения задаются независимо в расширенном диапазоне, позволяя создавать выраженную асимметрию пропускных способностей маршрутов. Пороговый параметр, управляющий автоматической логикой маршрутизации по второй полосе, дискретно варьируется в пределах целочисленного диапазона.

После генерации значения передаются серверу, инициируя немедленное обновление конфигурации. Каждое применение механизма создаёт уникальное сочетание параметров среды, формируя отдельный экспериментальный сценарий. Оператор, действующий в роли эксперта,

вынужден адаптировать свои решения к новым условиям, что обогащает корпус демонстрационных траекторий разнообразием контекстов и стратегий.

С позиции теории обучения с подкреплением данный подход обеспечивает широкое покрытие пространства состояний и параметров среды, что является необходимым условием для корректного восстановления функции вознаграждения методами обратного обучения. Формально данное требование соотносится с условием покрытия поддержки (support coverage): распределение состояний, посещённых экспертом в ходе демонстраций, должно иметь ненулевую плотность в каждой значимой области пространства состояний, в которой от агента может потребоваться принятие решения. Рандомизация параметров значительно расширяет поддержку эмпирического распределения демонстраций по сравнению с фиксированными параметрами среды, без необходимости ручного проектирования каждого отдельного сценария.

3.1.2 Механизм логирования и формат фиксируемых данных

Механизм логирования обеспечивает точную и своевременную фиксацию поведения экспертного агента в ходе взаимодействия с симуляционной средой. Данный механизм является ключевым связующим звеном между интерактивной симуляцией и последующим этапом обучения: именно накапливаемые журналы демонстраций составляют корпус обучающих данных, используемых алгоритмом обратного обучения с подкреплением.

Управление сбором данных реализовано через событийную модель взаимодействия клиента и сервера. Процедура инициализации логирования, запускаемая по команде оператора через интерфейс, переводит систему в активный режим записи: флаг активности устанавливается в истинное значение, буфер данных предыдущего сеанса очищается, и в отдельном фоновом потоке запускается процедура периодического опроса состояния. Разделение логирующего потока и основного симуляционного цикла

предотвращает взаимное торможение этих процессов и гарантирует отсутствие нарушений в работе симуляции в момент записи данных.

Периодический опрос выполняется с фиксированным интервалом в одну секунду, обеспечивая достаточную временную детализацию при умеренной нагрузке на систему хранения. Считывание значений наблюдаемых переменных выполняется потокобезопасно посредством примитивов взаимного исключения, что устраняет вероятность состояния гонки между симуляционным циклом, обновляющим параметры, и логирующим потоком, считывающим их.

Считанные значения формируют запись, которая дополняется временной меткой с миллисекундной точностью и помещается в накопительный буфер. По завершении сеанса логирования активный флаг сбрасывается, что приводит к корректному завершению логирующего потока. Немедленно после этого вызывается процедура сохранения, переносящая содержимое буфера в постоянное хранилище на диске. Процедура сохранения создаёт в рабочем каталоге приложения специализированный каталог для хранения экспертных демонстраций, если таковой ещё не существует, и записывает данные в файл формата CSV. Имя файла формируется автоматически на основе текущих даты и времени с точностью до секунды, что однозначно идентифицирует каждый сеанс записи, исключает конфликты имён при проведении серии экспериментов и обеспечивает естественную хронологическую сортировку файлов.

Каждый файл журнала представляет собой таблицу CSV, строки которой соответствуют отдельным моментам наблюдения с интервалом в одну секунду, а столбцы — компонентам вектора состояния среды и действиям агента. Выбор формата CSV обусловлен его универсальностью: данный формат нативно поддерживается всеми значимыми библиотеками для научных вычислений на языке Python, допускает непосредственный просмотр и редактирование в табличных редакторах и не требует специализированного программного обеспечения для чтения.

Первый столбец фиксирует временную метку наблюдения в формате «год-месяц-день часы:минуты:секунды.миллисекунды». Субсекундная точность позволяет проводить анализ временных паттернов с разрешением выше периода дискретизации, а также однозначно упорядочивать записи при объединении нескольких журналов в единый датасет.

Второй столбец содержит целочисленное значение текущей длины первой полосы движения — ключевую характеристику состояния транспортной сети, непосредственно наблюдаемую экспертным агентом. Значение принадлежит дискретному диапазону от нуля до максимальной вместимости очереди и отражает степень загруженности данного маршрута в момент наблюдения. Именно эта переменная является основным сигналом, на основании которого оператор принимает решение о допустимости приёма дополнительных транспортных единиц.

Третий столбец хранит вещественное значение параметра задержки разгрузки первой полосы движения, выражённого в секундах. Данный параметр характеризует текущую пропускную способность маршрута и является частью наблюдаемого контекста, необходимого для обоснованного принятия решений. Поскольку значение этого параметра может изменяться в течение сессии вследствие работы механизма случайной параметризации, его фиксация в каждый момент наблюдения обеспечивает корректную привязку действий к контексту их совершения.

Четвёртый и пятый столбцы содержат бинарные значения флагов маршрутизации для первой и второй полос движения. Флаг первой полосы представляет собой непосредственное решение оператора-эксперта: единица означает разрешение на приём, ноль — запрет. Именно этот столбец служит целевой переменной для алгоритма обратного обучения с подкреплением: задача модели состоит в восстановлении латентной функции вознаграждения, воспроизводящей наблюдаемую политику. Флаг второй полосы вычисляется автоматически по пороговому правилу и включается как контекстуальный

признак взаимозависимости маршрутов при восстановлении функции вознаграждения.

Выбор именно этих пяти переменных обусловлен принципом минимальной достаточности наблюдательного пространства. Избыточное расширение набора полей — например, включение длин всех трёх очередей, всех четырёх задержек и внутренних счётчиков — привело бы к экспоненциальному росту дискретного пространства состояний при табличной дискретизации и потребовало бы непропорционально большего объёма экспертных данных для статистически достоверных оценок. Чрезмерно компактный набор признаков не позволил бы алгоритму различить состояния, требующие принципиально различных решений. Компромисс — три информативных признака состояния и один коммуникативный сигнал соседнего агента — обеспечивает достаточную различающую способность при приемлемой размерности, что подтверждается экспериментальными результатами обучения.

Совокупность полей формирует минимально достаточный набор переменных для задачи обратного обучения с подкреплением: вектор наблюдений включает как количественные характеристики — заполненность очереди и пропускную способность, — так и бинарные дескрипторы управленческих решений. Каждая строка журнала соответствует тройке «состояние — действие — контекст» в формализме марковских процессов принятия решений. Файлы, накопленные в ходе множества сессий с различными параметрами среды, совместно составляют корпус экспертных демонстраций, достаточный для обучения модели, способной к обобщению на новые конфигурации транспортной сети.

3.2 Обучение и внедрение модели

3.2.1 Исходные данные и дискретизация пространства состояний

Основой для обучения модели обратного обучения с подкреплением послужили экспертные траектории, собранные в ходе работы многоагентной транспортной симуляции. Экспертные данные представляют собой записи о последовательных решениях управляющего агента `highway1` в различных конфигурациях дорожной среды. Каждая запись датасета содержит набор признаков, описывающих текущее состояние системы в момент наблюдения, а также целевую переменную — фактическое решение агента-эксперта.

В обучающую выборку включены следующие информативные поля: число автомобилей в очереди `highway1` (`highway1_len`), пропускная способность полосы в виде задержки выезда (`drain_delay_h1`) и сигнал готовности `highway2` к приёму автомобиля (`can_take2`). Целевой переменной являлось бинарное решение `highway1` о собственной готовности принять транспортное средство (`can_take1`). Для построения матрицы переходов использовались соответствующие значения следующего шага: `next_highway1_len`, `next_drain_delay_h1` и `next_can_take2`. Суммарная размерность признакового пространства составила шесть переменных.

Поля `timestamp` и `step` были исключены из процесса обучения, поскольку выполняют исключительно идентификационную функцию: не несут содержательной информации о состоянии транспортной среды. Включение подобных полей привело бы к нежелательной зависимости выученной функции вознаграждения от абсолютного положения во времени, что противоречит принципу стационарности марковского процесса принятия решений.

Включение признаков следующего шага играет ключевую роль при построении матрицы переходов $T(s'|s,a)$. Каждый обучающий пример явно кодирует как исходное состояние s , так и состояние-преемник s' , что позволяет алгоритму восстанавливать динамику среды непосредственно из данных, не прибегая к дополнительным предположениям о структуре переходов. При отсутствии признаков следующего шага матрицу переходов пришлось бы оценивать косвенно — по порядку следования записей в траектории.

Подобный подход существенно менее надёжен: пропуски, дубликаты или нарушения временного порядка непосредственно искажали бы оценку $T(s'|s,a)$. Явное указание целевого состояния устраняет эту зависимость и делает оценку устойчивой к артефактам сбора данных, что особенно важно при работе с реальными журналами транспортных систем.

Постановка задачи обратного обучения с подкреплением в данном контексте состоит в следующем: по трём наблюдаемым характеристикам текущего состояния среды необходимо восстановить скрытую функцию вознаграждения, которой руководствовался агент-эксперт при выработке решений. Предполагается, что наблюдаемое поведение является оптимальным или близким к оптимальному, и задача алгоритма — аппроксимировать эту функцию так, чтобы воспроизводимое поведение максимально соответствовало экспертному.

Метод максимально-энтропийного обратного обучения с подкреплением в табличной постановке (Tabular MCE IRL) предполагает работу с конечным и счётным пространством состояний. Поскольку часть признаков принимает непрерывные значения, необходим этап дискретизации, переводящий вещественные векторы наблюдений в целочисленные индексы состояний.

Дискретизация выполнялась с использованием следующей стратегии. Для каждого признака предварительно подсчитывалось число уникальных значений, встречающихся в обучающей выборке. Если число уникальных значений не превышало установленного порога $MAX_BINS = 4$, признак сохранял свою естественную структуру без дополнительного огрубления — это позволяет точнее отразить реальное распределение данных в тех случаях, когда признак уже де-факто является дискретным или принимает лишь небольшое число различных значений. Если же признак оказывался непрерывным и принимал большое число значений, он разбивался на перцентильные интервалы равной наполненности, каждый из которых соответствовал одному из MAX_BINS дискретных состояний. Применение

перцентильного разбиения, в отличие от равномерного разбиения диапазона значений, обеспечивает равномерную населённость каждого бина, что является важным условием статистической состоятельности оценки меры посещаемости состояний. Бинарные поля, такие как `can_take2` и `can_take1`, дискретизировались тривиально, поскольку их область значений уже образует минимально возможное конечное множество.

После дискретизации каждый вектор признаков однозначно отображался в целочисленный индекс посредством кодирования смешанного основания: индекс вычислялся как взвешенная сумма дискретных значений с весами, соответствующими произведениям мощностей областей предшествующих признаков. Это обеспечивает биективное соответствие между кортежами дискретизированных значений и целочисленными индексами. Итоговая мощность дискретного пространства состояний составила 576 состояний, что достаточно для представления разнообразия конфигураций дорожной системы, зафиксированных в экспертных траекториях. Данный порядок величины является компромиссным: при существенно меньшем числе состояний (порядка десятков) различные конфигурации среды сливались бы в единое состояние, лишая алгоритм возможности различать ситуации, требующие принципиально различных решений. При существенно большем числе состояний (порядка десятков тысяч) матрица переходов становилась бы крайне разреженной, а оценки вероятностей переходов — статистически ненадёжными при имеющемся объёме данных. Выбранная размерность пространства состояний обеспечивает достаточную гранулярность для различения значимых конфигураций среды при сохранении статистической достоверности табличных оценок.

3.2.2 Построение модели переходов и вычисление меры посещаемости

Матрица переходов является ключевым компонентом табличной модели марковского процесса принятия решений и определяет динамику среды: вероятность перехода из текущего состояния в каждое из возможных следующих состояний при заданном действии. В данной работе матрица восстанавливалась непосредственно из экспертных данных методом максимального правдоподобия.

Для каждого последовательного шага в экспертных траекториях фиксировалась тройка: текущее состояние s_t , совершённое действие a_t и результирующее состояние s_{t+1} . Счётчик переходов $T[s_t, a_t, s_{t+1}]$ инкрементировался для каждой тройки. Матрица формировалась как трёхмерный тензор размерности (S, A, S) , где $S = 576$, $A = 2$.

Для обеспечения числовой корректности алгоритма необходимо, чтобы матрица переходов была полностью заполнена и стохастической по третьему измерению. В частности, для состояний, которые не встречались в обучающей выборке ни разу, или для пар (состояние, действие) без наблюдений, счётчики оказывались нулевыми. Во избежание деления на ноль к нулевым строкам добавлялся малый равномерный сдвиг (лапласовское сглаживание), распределяющий вероятность равномерно по всем возможным следующим состояниям. После сглаживания каждая строка нормализовалась до единичной суммы, гарантируя корректность марковской модели.

Помимо матрицы переходов, для каждого дискретного состояния вычислялся средний вектор наблюдений: для всех записей датасета, отображённых в данное состояние, усреднялись исходные вещественнозначные значения признаков. Полученная матрица размерностью $(S, \text{obs_dim})$ позволяет ставить в соответствие каждому дискретному индексу интерпретируемое описание в исходном признаковом пространстве, а также служит входными данными для нейросетевой аппроксимации вознаграждения.

Центральным понятием метода MCE IRL является мера посещаемости состояний (state occupancy measure) — вероятностное распределение над

пространством состояний, отражающее относительную частоту нахождения агента в каждом состоянии при следовании наблюдаемой политике на протяжении горизонта планирования. Состояния, которые эксперт посещает чаще, получают более высокий вес, и именно воспроизведение этого распределения служит основной целью оптимизации MCE IRL.

Мера посещаемости эксперта вычислялась непосредственно из экспертных траекторий следующим образом. Для каждой записи датасета определялся целочисленный индекс состояния посредством описанной выше процедуры дискретизации. Затем строился вектор счётчиков посещений, подсчитывающий число попаданий в каждый из 576 дискретных состояний. Результирующий вектор нормализовался делением на суммарное число записей, что переводило абсолютные счётчики в относительные частоты и обеспечивало выполнение аксиом вероятностного распределения. Итоговый вектор размерностью (S_s) представлял собой оценку меры посещаемости эксперта по методу максимального правдоподобия.

Формальная связь между мерой посещаемости и политикой агента состоит в следующем: если агент следует стационарной марковской политике $\pi(a|s)$ в среде с начальным распределением состояний μ_0 и матрицей переходов $T(s'|s,a)$, то мера посещаемости определяется как суммарная дисконтированная вероятность нахождения в состоянии s на протяжении всего горизонта. Алгоритм MCE IRL ищет функцию вознаграждения, при которой оптимальная политика агента порождает меру посещаемости, совпадающую с экспертной. Это условие согласования мер посещаемости является операциональным определением «объяснения» экспертного поведения в рамках данного подхода.

3.2.3 Обучение функции вознаграждения и извлечение политики

Функция вознаграждения аппроксимировалась нейронной сетью класса BasicRewardNet из библиотеки imitation. Архитектура сети включала один скрытый слой с 64 нейронами и нелинейной функцией активации.

Вознаграждение зависело исключительно от текущего состояния среды: параметры конфигурации гарантировали, что оно не обуславливалось ни совершённым действием, ни результирующим состоянием, ни признаком завершённости эпизода. Входом сети служил вектор средних наблюдений, выходом — скалярное значение $R(s)$.

Обучение проводилось посредством алгоритма MCEIRL в течение 100 итераций с коэффициентом обучения 0.01 и коэффициентом дисконтирования $\gamma = 0.99$. Каждая итерация включала три последовательных этапа. На первом этапе по текущим весам сети вычислялось вознаграждение для каждого из 576 состояний, формируя вектор R . На втором этапе решалась прямая задача оптимального управления: методом обратной индукции по горизонту планирования ($horizon = 139$) находилась оптимальная мягкая политика. Мягкое уравнение Беллмана имеет вид $Q(s,a) = R(s) + \gamma \cdot \sum_{s'} T(s,a,s') \cdot V(s')$, где функция ценности $V(s)$ вычислялась через log-sum-exp оператор, что соответствует максимально-энтропийной постановке. На третьем этапе по найденной политике вычислялась мера посещаемости агента, и корректировались параметры нейронной сети в направлении уменьшения расхождения с экспертной мерой.

Максимально-энтропийный подход ставит задачу восстановления вознаграждения как максимизацию энтропии распределения над траекториями при ограничении на совпадение ожидаемых признаков с экспертными. Это обеспечивает единственность решения и устойчивость к неоднозначности, присущей классическим методам IRL, где одному поведению могут соответствовать бесконечно много функций вознаграждения. Нейросетевой аппроксиматор позволяет моделировать нелинейные зависимости между признаками и вознаграждением, что особенно важно в транспортных задачах с неочевидными взаимодействиями переменных.

Градиент логарифмического правдоподобия в точности равен разности между частотами посещения состояний экспертом и обучаемой политикой. По

мере обучения эта разность стремится к нулю, что служит естественным критерием сходимости. Значение $\gamma = 0.99$ отражает свойства транспортной задачи: последствия маршрутизации проявляются в виде заторов и задержек на протяжении многих последующих шагов, и высокое значение γ обеспечивает учёт долгосрочных эффектов наравне с немедленными, не позволяя алгоритму вырождаться в жадную стратегию, ориентированную только на сиюминутное вознаграждение. Мониторинг нормы градиента в ходе обучения позволяет своевременно обнаруживать стагнацию и при необходимости корректировать шаг градиентного спуска.

По завершении обучения из функции вознаграждения извлекалась детерминированная политика принятия решений. Для каждого состояния вычислялись Q-значения для каждого из двух действий, по которым строилось распределение вероятности. Таблица политики имела размерность (576, 2). Для практического применения использовалось пороговое правило: если вероятность действия `can_take1 = 1` превышала 0.5, агент принимал положительное решение.

Полученное значение accuracy составило ~ 0.295 , что требует контекстной интерпретации. При высокой размерности пространства состояний (576 состояний) и ограниченном горизонте (139 шагов) большинство состояний посещается экспертом ноль или один раз, что делает поточечное сравнение действий статистически ненадёжным. Политика MCE IRL является стохастической мягкой политикой, назначающей ненулевую вероятность обоим действиям; пороговое отсечение при 0.5 огрубляет непрерывное распределение до бинарной метки. Целевая функция MCE IRL — совпадение мер посещаемости, а не точность предсказания отдельных действий — эти метрики оптимизируют принципиально разные свойства политики. Наконец, поведение самого эксперта может быть непоследовательным в схожих состояниях, что образует теоретический потолок точности ниже единицы.

3.2.4 Сохранение и применение модели

По завершении обучения все компоненты модели сохранялись в двух файлах, предназначенных для независимого хранения весовых параметров и метаданных.

Первый файл, `mse_irl_reward.pt`, сохранялся в формате PyTorch и содержал полный набор данных, необходимых для воспроизведения инференса: веса нейронной сети, таблицу политики (S, A), матрицу переходов (S, A, S), матрицу средних наблюдений ($S, \text{obs_dim}$), вектор начального распределения и гиперпараметры обучения — коэффициент дисконтирования, горизонт планирования и число итераций. Хранение всех компонентов в едином файле гарантирует консистентность модели при переносе и воспроизведении результатов.

Второй файл, `mse_irl_meta.json`, содержал метаданные: список входных признаков с именами и порядком следования, достигнутую точность на обучающей выборке, размерность дискретного пространства состояний, число действий и параметры дискретизации для каждого признака — пороговые значения перцентильных границ или перечень уникальных значений. Наличие метаданных в открытом формате позволяет воспроизвести процедуру дискретизации на новых данных без необходимости повторного запуска обучающего кода.

Разделение артефактов модели на два файла — бинарный и текстовый — преследует цель обеспечения как эффективности загрузки, так и прозрачности параметров для исследователя. Бинарный формат PyTorch обеспечивает компактность хранения и быструю десериализацию при загрузке модели; текстовый JSON-формат позволяет визуально инспектировать параметры обучения без запуска кода, что упрощает аудит и сравнение различных экспериментальных конфигураций. Данный подход к организации артефактов обучения соответствует современным практикам воспроизводимых исследований в области машинного обучения.

Применение модели к новым наблюдениям имеет минимальную вычислительную стоимость. Новый вектор признаков состояния, поступающий из симуляции, последовательно дискретизируется по сохранённым параметрам дискретизации: каждый непрерывный признак отображается в соответствующий бин по пороговым значениям, а результирующий кортеж дискретных значений преобразуется в целочисленный индекс состояния по схеме кодирования смешанного основания. Полученный индекс используется для непосредственного обращения к таблице политики, из которой извлекается вероятность действия $\text{can_take1} = 1$. Описанная процедура инференса имеет временную сложность $O(1)$ относительно числа состояний и числа итераций обучения, что делает модель пригодной для применения в режиме реального времени в рамках многоагентной симуляции. Нейронная сеть при инференсе не используется: её роль завершается на этапе обучения, где она определяет вознаграждение для каждого состояния и формирует итоговую таблицу политики.

3.2.5 Интеграция агента в систему и замена ручного управления

Завершающим этапом разработки многоагентной симуляционной среды является интеграция обученного агента, ядром которого служит модель MSE IRL, в функционирующую симуляционную систему. Данный этап представляет собой переход от автономного обучения модели к её практическому применению в режиме реального времени, что является ключевым требованием для демонстрации работоспособности предложенного подхода.

Архитектурно интеграция реализована по принципу минимального вмешательства: модель встраивается в симуляционный цикл так, что все прочие компоненты — механизм обмена состояниями, веб-интерфейс, протокол трансляции — сохраняют структуру без принципиальных изменений. Единственным преобразованием подвергается участок кода, где вызов пользовательского ввода замещается вызовом процедуры инференса.

Такая стратегия интеграции обладает рядом методологических преимуществ. Во-первых, она обеспечивает воспроизводимость экспериментов: поведение системы до и после замены ручного управления автономным агентом остаётся сопоставимым, что позволяет проводить корректную оценку качества принимаемых моделью решений. Во-вторых, изолированность модуля инференса упрощает его верификацию и возможную замену альтернативными моделями без переработки всей системы. В-третьих, сохранение единого симуляционного цикла исключает необходимость в дополнительных механизмах синхронизации между подсистемой управления и подсистемой моделирования.

В прототипной версии симуляционной системы параметр `can_take1` устанавливался оператором посредством интерактивного элемента управления в веб-интерфейсе. Данная схема соответствовала задачам начального этапа исследования, когда требовалось накопить демонстрационные траектории для последующего обучения. Однако для финальной версии системы подобная схема принципиально неприемлема. В итоговой реализации кнопка ручного управления полностью исключена из интерфейса. На уровне серверной логики введена защита параметра от внешней модификации: обработчик входящих сообщений блокирует любые попытки установить значение `can_take1` извне. Аналогичной защитой снабжены параметры `can_take1_prob` и `running`. В обработчик введена система логирования попыток несанкционированного доступа: каждый отвергнутый запрос фиксируется в серверном журнале с указанием источника и запрашиваемого значения, позволяя диагностировать ошибки интеграции и верифицировать, что автономия модели не была скомпрометирована.

Данное архитектурное решение реализует принцип единственного источника истины: значение `can_take1` в любой момент времени является исключительно результатом работы модели, а не артефактом внешнего вмешательства. Это принципиально важно для обеспечения корректности экспериментальных измерений и воспроизводимости наблюдаемого

поведения. Передача управления автономному агенту полностью устраняет неопределённость, связанную с человеческим фактором, и гарантирует, что наблюдаемые паттерны поведения однозначно атрибутируются политике, извлечённой из демонстрационных данных методом MCE IRL. Разграничение полномочий между пользователем и автономным агентом отражает фундаментальное различие между режимом сбора данных и режимом автономного управления и является необходимым условием корректной интерпретации результатов моделирования.

Описанный паттерн интеграции демонстрирует, что агент, обученный методом IRL, может быть бесшовно подставлен на место оператора-человека, валидируя сквозной конвейер: от сбора демонстраций через обучение вознаграждения к автономному развёртыванию. Каждый этап конвейера реализован как независимый модуль с определёнными интерфейсами, что облегчает замену компонентов и обобщение на другие предметные области, в которых экспертное поведение доступно для наблюдения, однако явная формализация вознаграждения затруднена: управление воздушным движением, планирование логистических цепочек, распределение вычислительных ресурсов.

3.2.6 Загрузка модели и инференс в симуляционном цикле

Процедура загрузки обученной модели реализована на уровне инициализации серверного модуля и выполняется однократно при запуске приложения, до начала обработки клиентских запросов. Такой подход исключает задержки инференса и обеспечивает готовность агента с первого шага симуляции.

Модель представлена двумя файлами: файлом весов в формате PyTorch (.pt) и файлом метаданных в формате JSON (.json). Оба файла формируются на этапе обучения в прототипной подсистеме и передаются в основную систему через файловую систему, обеспечивая воспроизводимость и портативность модели.

Процедура загрузки предусматривает проверку существования обоих файлов. При успехе производится десериализация весов и чтение метаданных; в консоль выводится информация о точности модели для верификации загруженного артефакта. Результат загрузки сохраняется как глобальный объект модуля и используется всеми последующими вызовами процедуры инференса.

Принципиально важно наличие резервного режима на случай отсутствия файлов модели. Если один или оба файла недоступны, система не прерывает инициализацию с ошибкой, а переходит в безопасный режим: параметр `can_take1` фиксируется в значении 1 (разрешение приёма), а в журнал системы выводится предупреждение о недоступности модели. Данное решение обеспечивает непрерывность работы симуляции даже в случае нештатных ситуаций при развёртывании, что соответствует требованиям к отказоустойчивости систем реального времени. Значение по умолчанию, равное 1, выбрано как консервативная стратегия, минимизирующая риск простоя моделируемой транспортной сети.

Инференс модели осуществляется в рамках функции предсказания, вызываемой на каждом шаге симуляционного цикла в момент обработки события передачи транспортной единицы. Функция принимает три входных аргумента, соответствующих наиболее информативным признакам состояния среды, выявленным в процессе анализа демонстрационных данных: длину очереди на первом участке трассы, задержку дренажа на первом узле и текущее значение разрешающего флага второго узла сети.

Внутренняя логика функции реализует полный конвейер преобразования непрерывных наблюдений в дискретное пространство состояний. На первом шаге каждый входной признак дискретизируется в соответствии со схемами, сохранёнными в метаданных модели: для каждого признака определяются границы интервалов, и текущее значение сопоставляется с соответствующим дискретным бином. Данный шаг воспроизводит процедуру дискретизации, применявшуюся при формировании

обучающей выборки, что обеспечивает согласованность пространства состояний между фазой обучения и фазой применения модели.

На втором шаге из комбинации дискретных значений признаков формируется индекс состояния, однозначно идентифицирующий текущую конфигурацию среды в табличном пространстве состояний. По данному индексу из таблицы стохастической политики извлекается вероятность действия «разрешить приём» (значение `can_take1 = 1`). Итоговое бинарное решение формируется путём сравнения извлечённой вероятности с порогом 0.5: при вероятности, превышающей порог, агент принимает решение о допуске транспортной единицы; в противном случае — об отказе.

Результаты инференса — бинарное решение и вероятность — записываются в конфигурационный объект симуляции под защитой мьютекса блокировки. Использование блокировки является обязательным в многопоточной архитектуре: симуляционный цикл и обработчики клиентских запросов выполняются в разных потоках, и без синхронизации возможно состояние гонки при одновременном чтении и записи разделяемых данных. Атомарность записи гарантирует, что клиенты никогда не получат несогласованное состояние, в котором решение и вероятность относятся к разным шагам.

3.2.7 Аппроксимация будущего состояния и отображение решения в интерфейсе

Одной из ключевых особенностей, требующих специального рассмотрения при переходе от обучения к применению модели, является расхождение между признаковым пространством обучающей выборки и информацией, доступной в реальном времени. Модель MCE IRL обучена на признаках, включающих как значения текущего шага, так и значения следующего шага (поля с префиксом `next_*`). Однако в условиях реального функционирования симулятора истинные значения следующего шага

принципиально недоступны в момент принятия решения: они станут известны лишь после выполнения шага симуляции.

Для разрешения указанного противоречия в системе принято следующее аппроксимирующее допущение: значения признаков будущего состояния полагаются равными текущим значениям соответствующих переменных. Иными словами, при формировании вектора признаков для инференса поля `next_highway1_len`, `next_drain_delay_h1` и `next_can_take2` заполняются теми же значениями, что и соответствующие текущие поля. Данная аппроксимация основана на предположении о медленном изменении параметров системы: за один шаг может произойти не более одного элементарного события — добавление или удаление одного автомобиля, что ограничивает максимальное изменение `highway1_len` единицей. Параметры задержки изменяются лишь при явном вмешательстве оператора или срабатывании механизма рандомизации, что происходит значительно реже интервала дискретизации. Бинарный сигнал `can_take2` определяется пороговой функцией и изменяется лишь при пересечении порогового значения. Анализ демонстрационных данных подтвердил, что абсолютные изменения ключевых параметров между соседними шагами в подавляющем большинстве случаев не превышают одного дискретного интервала. Следовательно, при дискретизации признаков, применяемой в процедуре инференса, аппроксимация будущего состояния текущим не приводит к систематическому смещению индекса состояния, а вносимая погрешность ограничена шириной одного интервала дискретизации. Таким образом, применяемое допущение является методологически обоснованным и не подрывает валидность принимаемых агентом решений.

Переход от ручного управления к автономному агенту потребовал адаптации пользовательского интерфейса. Поскольку `can_take1` более не является управляемой пользователем переменной, интерактивный элемент управления — кнопка переключения — преобразован в пассивный визуальный индикатор, доступный исключительно для чтения.

Индикатор отображает два вида информации: бинарное решение агента (0 или 1, соответствующее запрету или разрешению приёма) и численную вероятность в процентах. Для идентификации источника решения индикатор снабжён меткой «MCE IRL». Визуальное состояние кодируется стилевыми классами: один класс соответствует положительному решению, другой — отрицательному, обеспечивая интуитивное восприятие состояния системы.

Синхронизация отображаемых значений на всех подключённых клиентах обеспечивается механизмом широковещательной рассылки: каждое сообщение содержит актуальные значения как часть единого конфигурационного словаря. Тем самым гарантируется, что все наблюдатели, подключённые к системе в данный момент, видят идентичное и согласованное состояние агента вне зависимости от момента подключения.

Подобное оформление несёт как практическую, так и методологическую функцию: наглядно демонстрирует факт замены ручного управления автономным агентом и позволяет наблюдать процесс принятия решений в динамике симуляции. Отображение вероятностной оценки, а не только бинарного решения, даёт дополнительный инструмент интерпретации: вероятности вблизи 0.5 сигнализируют о неопределённости агента и могут служить индикатором переходных или нетипичных состояний среды, представляющих интерес для анализа обобщающей способности модели.

3. 3 Оценка результатов экспериментов

Тестирование разработанной системы проводилось на нескольких уровнях: верификация компонентов симуляционной среды, проверка корректности обучения модели и валидация поведения интегрированного агента в условиях реального функционирования.

На первом этапе выполнялась верификация симуляционного цикла. Проверялось соблюдение инвариантов среды: длина каждой очереди не превышает установленной ёмкости, дисциплина обслуживания FIFO

сохраняется при любых конфигурациях параметров, временные задержки корректно регулируют интенсивность событий. Тестирование проводилось путём запуска симуляции с граничными значениями параметров: минимальной и максимальной интенсивностью входящего потока, нулевой и предельной заполненностью очередей. Во всех случаях система сохраняла устойчивое функционирование без нарушения ограничений.

На втором этапе верифицировался механизм сбора экспертных демонстраций. Проверялась полнота и корректность записываемых данных: соответствие значений в журнале фактическому состоянию симуляции, отсутствие пропусков и дублирования записей, корректность временных меток. Потокбезопасность механизма логирования тестировалась при одновременной работе симуляционного цикла и логирующего потока в условиях частого изменения параметров среды.

На третьем этапе оценивалась корректность процедуры обучения модели. Контролировалось соответствие размерности дискретного пространства состояний ожидаемому значению (576 состояний при заданных параметрах дискретизации). Проверялась стохастичность матрицы переходов: сумма вероятностей по каждой строке равна единице, отсутствуют нулевые строки благодаря лапласовскому сглаживанию. Таблица политики верифицировалась на предмет нормализации: сумма вероятностей действий для каждого состояния равна единице.

На четвёртом этапе проводилось тестирование интеграции обученного агента в симуляцию. Верифицировалась корректность загрузки артефактов модели при инициализации сервера, а также работоспособность резервного режима при отсутствии файлов модели. Проверялась защита параметра решения агента от внешней модификации: попытки изменения через пользовательский интерфейс отклонялись серверной логикой и фиксировались в журнале. Тестировалась согласованность отображаемых в интерфейсе значений решения и вероятности с фактическими результатами инференса.

Заключительный этап включал функциональное тестирование системы в целом. Симуляция запускалась с различными конфигурациями параметров среды, включая сгенерированные механизмом рандомизации. Контролировалось, что обученный агент принимает решения на каждом шаге без задержек и сбоев, визуализация корректно отражает текущее состояние, а многопоточная архитектура обеспечивает отсутствие состояний гонки при одновременном обращении к разделяемым данным. Отдельно проверялась синхронизация данных между сервером и клиентскими приложениями: при подключении нескольких браузерных клиентов одновременно все получали идентичное и согласованное представление состояния системы.

Помимо перечисленных проверок, проводилось сравнительное тестирование поведения системы в двух режимах: ручного управления оператором и автономного управления обученным агентом. При идентичных начальных условиях и параметрах среды фиксировались динамика заполненности очередей и частота переключений решения о маршрутизации. Результаты подтвердили, что обученный агент воспроизводит качественные паттерны экспертного поведения: реагирует на рост заполненности ограничением приёма и учитывает сигнал готовности второго агента при формировании решения.

Совокупность проведённых тестов охватывает все ключевые компоненты системы — от низкоуровневых инвариантов структур данных до высокоуровневого поведения интегрированного агента — и подтверждает корректность функционирования разработанного программного обеспечения.

ЗАКЛЮЧЕНИЕ

В рамках настоящей магистерской диссертации исследована задача моделирования поведения агентов в многоагентной среде с применением методов обратного обучения с подкреплением.

На первом этапе проведён анализ существующих подходов к управлению в многоагентных системах. Рассмотрены как классические неинтеллектуальные методы (фиксированные расписания, статические правила), так и современные интеллектуальные подходы на основе обучения с подкреплением. Обоснован выбор метода Maximum Causal Entropy IRL (MCE IRL) в качестве основного алгоритма восстановления функции вознаграждения по экспертным демонстрациям. Данный метод обеспечивает устойчивость к шуму в демонстрациях и позволяет получать стохастические политики с максимальной энтропией, что является существенным преимуществом в условиях неопределённости многоагентной среды.

На втором этапе спроектирована и реализована многоагентная симуляционная среда, моделирующая транспортную систему с двумя взаимодействующими агентами. Среда обладает ключевыми свойствами, характерными для реальных многоагентных систем: частичная наблюдаемость (каждый агент располагает только локальной информацией), взаимозависимость решений (действия одного агента влияют на состояние другого) и бинарная коммуникация между агентами.

На третьем этапе реализован полный конвейер обратного обучения с подкреплением: сбор экспертных демонстраций, обучение модели методом MCE IRL, восстановление функции вознаграждения и интеграция обученного агента в симуляционную среду. Обученная модель автономно принимает решения о маршрутизации транспортных потоков, заменяя ручное управление. Экспериментальная проверка подтвердила, что агент, обученный на восстановленной функции вознаграждения, воспроизводит поведение эксперта в рамках заданной среды.

Таким образом, поставленные в работе задачи решены в полном объеме. Полученные результаты демонстрируют применимость метода MCE IRL для моделирования поведения агентов в многоагентных средах с частичной наблюдаемостью и ограниченной коммуникацией.

СПИСОК ЛИТЕРАТУРЫ

1. Adams, S., Cody, T., Beling, P. A. A Survey of Inverse Reinforcement Learning: Challenges, Methods and Progress // Artificial Intelligence. — 2022. — Vol. 297. — Article 103500. — DOI: 10.1016/j.artint.2021.103500.
2. Yu, L., Song, J., Ermon, S. Multi-Agent Adversarial Inverse Reinforcement Learning // Proceedings of the 36th International Conference on Machine Learning (ICML 2019), republished in: Journal of Machine Learning Research. — 2021. — Vol. 22, No. 1. — . 1–34. — DOI: 10.48550/arXiv.1907.13220.
3. Arenz, O., Zhong, M., Neumann, G. Non-Adversarial Imitation Learning and Its Connections to Adversarial Methods // Proceedings of the 25th International Conference on Artificial Intelligence and Statistics (AISTATS 2022). — PMLR. — 2022. — Vol. 151. — P. 1008–1024.
4. Snoswell, A. J., Singh, S. P., Ye, N. Revisiting Maximum Entropy Inverse Reinforcement Learning: New Perspectives and Algorithms // Proceedings of the AAAI Conference on Artificial Intelligence. — 2023. — Vol. 37, No. 8. — P. 9939–9947. — DOI: 10.1609/aaai.v37i8.26178.
5. Terry, J. K., Black, B., Grammel, N. et al. PettingZoo: Gym for Multi-Agent Reinforcement Learning // Advances in Neural Information Processing Systems (NeurIPS 2021). — 2021. — Vol. 34. — P. 15032–15043.